FIAP · 2TDS · FASE 6 — NAVEGANDO PELO MUNDO DEVOPS

Cidades ESGInteligentes

API de Gestão de Resíduos e Reciclagem com pipeline CI/CD, Docker e deploy automatizado em staging e produção

Integrante	**Guilherme Eger Pereira** — RM 97069
Projeto base	API C# (.NET 8) desenvolvida na Fase 4 — tema ESG: gestão de resíduos
Repositório	https://github.com/GuilhermeEger/cidades-esg-inteligentes
Execução de referência	https://github.com/GuilhermeEger/cidades-esg-inteligentes/actions/runs/38284813510 (commit `e4b978c`)
Ferramentas	GitHub Actions · Docker · Docker Compose · GHCR · SQL Server 2022
Data	12/10/2026

1. Visão geral

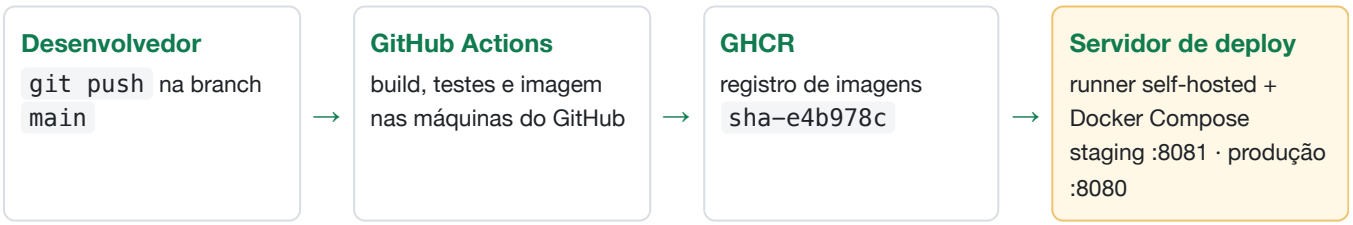
A **Cidades ESGInteligentes** é uma API REST que apoia a gestão de resíduos de uma cidade inteligente: cadastra pontos de coleta, registra coletas por tipo de resíduo (papel, plástico, vidro, metal, orgânico e eletrônico) e calcula indicadores de reciclagem. Ela foi construída na Fase 4 em ASP.NET Core 8 com Entity Framework Core e SQL Server.

Nesta fase, o objetivo foi **automatizar todo o ciclo de vida** da aplicação com práticas DevOps: cada alteração enviada ao GitHub é compilada, testada, empacotada em uma imagem Docker e publicada automaticamente em **staging**; depois de uma aprovação manual, a mesma imagem vai para **produção**.

Adaptações para simular produção

Melhoria na aplicação	Motivo
`GET /health` com verificação do banco	Usado pelo HEALTHCHECK do Docker, pelo `docker compose --wait` e pelos smoke tests
`GET /` com ambiente e versão	Mostra qual commit está no ar em cada ambiente (rastreabilidade)
`appsettings.Staging.json` e `appsettings.Production.json`	Logs e Swagger diferentes por ambiente; Swagger desligado em produção
Segredos apenas por variáveis de ambiente	Senha do banco, chave JWT e senha da API vêm dos *secrets* do GitHub
Retry de conexão com o banco (EF Core)	A API tolera o SQL Server demorar a subir ou oscilar
+2 testes automatizados (total de 9)	Cobrem o health check e o endpoint de versão

Arquitetura de entrega

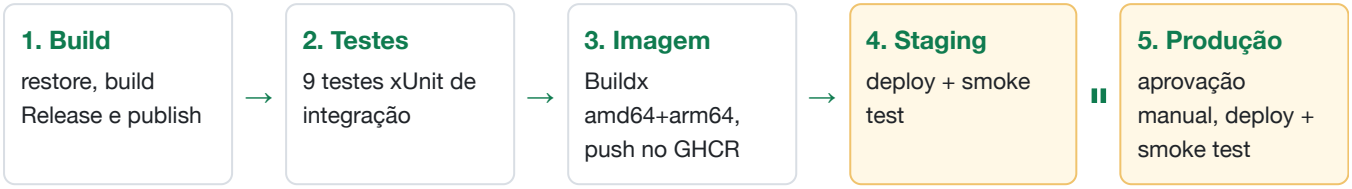


O servidor de deploy é um Mac com Docker (Colima) registrado como *self-hosted runner* do repositório: o GitHub envia a ele os jobs de deploy, que baixam a imagem do GHCR e sobem os containers.

> **Onde os ambientes estão hospedados.** Staging e produção rodam em containers Docker reais no computador do aluno, que faz o papel de servidor de deploy, e são acessados em `localhost` (8081 e 8080). É uma simulação de produção sem custo de nuvem: em uma empresa, o mesmo runner seria instalado em um servidor ou VM na nuvem (por exemplo Azure ou AWS) com um domínio público, e o pipeline continuaria exatamente o mesmo.

2. Pipeline CI/CD

Ferramenta: GitHub Actions — arquivo `.github/workflows/ci-cd.yml`. O pipeline tem 5 jobs encadeados com `needs`: se um falhar, os seguintes não rodam.



Job	Onde roda	O que faz
1. Build	ubuntu-24.04	`dotnet restore`, `dotnet build -c Release` e `dotnet publish`; o pacote é guardado como artefato da execução.
2. Testes automatizados	ubuntu-24.04	`dotnet test` com xUnit + WebApplicationFactory + EF InMemory (não precisa de banco). O resultado vira uma tabela no resumo do job e o relatório `.trx` fica como artefato.
3. Imagem Docker (GHCR)	ubuntu-24.04	Build multi-arquitetura com Docker Buildx e push para o GitHub Container Registry com as tags `sha-<commit>` e `latest`. Usa cache de camadas do GitHub Actions.
4. Deploy Staging	self-hosted	Login no GHCR, `deploy/deploy.sh staging <imagem>` (Docker Compose, porta 8081) e `deploy/smoke-test.sh`.
5. Deploy Produção	self-hosted	Só inicia após **aprovação manual** no Environment `production`. Publica a **mesma imagem** de staging na porta 8080 e roda o smoke test.

Lógica do pipeline

Decisão	Como funciona
Gatilhos	`push` na `main` → fluxo completo. `pull_request` → só build e testes (valida sem publicar). `workflow_dispatch` → execução manual. Mudanças só em documentação (`docs/`, `*.md`) não disparam o pipeline.
Build once, deploy many	A imagem é criada uma única vez e promovida de staging para produção. A tag `sha-<commit>` e o endpoint `GET /` provam que é a mesma versão.
Environments do GitHub	`staging` e `production`, cada um com seus *secrets* (`DB_PASSWORD`, `JWT_SECRET`, `API_PASSWORD`). `production` exige revisor.
Smoke test pós-deploy	Health check (API + banco), ambiente/versão, leitura de tipos de resíduo e indicadores, login JWT, rota protegida com token (200) e sem token (401). Qualquer falha marca o job como falho.
Segurança	Permissões mínimas do `GITHUB_TOKEN` por job; deploy nunca roda em pull request; PRs de pessoas externas exigem aprovação; logout do registry ao final.

3. Docker: arquitetura, comandos e imagem

Arquitetura dos containers

O mesmo `docker-compose.yml` atende local, staging e produção. Cada ambiente é um **projeto Compose** separado (`-p residuos-staging` / `-p residuos-production`), com containers, rede e volume próprios.

Recurso	Configuração
Serviços	`api` (imagem do projeto, porta 8080 no container) e `db` (SQL Server 2022)
Volume	`db-data` → `/var/opt/mssql`: os dados do banco sobrevivem a novos deploys
Redes	`backend` **interna** (só API ↔ banco; o banco não é exposto) e `frontend` (porta publicada da API)
Variáveis de ambiente	Local: `.env` (copiado de `.env.example`). Deploy: `deploy/<ambiente>.env` (porta, nome do banco, Swagger) + secrets do GitHub. Obrigatórias declaradas como `${VAR:?erro}`
Saúde e ordem de subida	Healthcheck no banco (`sqlcmd`) e na API (`/health`); `depends_on: service_healthy`; deploy com `--wait`
Resiliência	`restart: unless-stopped` nos dois serviços

	Staging	Produção
URL	http://localhost:8081	http://localhost:8080
ASPNETCORE_ENVIRONMENT	Staging	Production
Banco	ResiduosDb_Staging (container próprio)	ResiduosDb (container próprio)
Swagger	ligado	desligado
Publicação	automática após os testes	após aprovação manual

Imagem criada

Registro	GitHub Container Registry (GHCR)
Imagem	`ghcr.io/guilhermeeger/cidades-esg-inteligentes:sha-e4b978c` (também `latest`)
Plataformas	linux/amd64 e linux/arm64
Tamanho (arm64, no servidor)	378MB
Base	`mcr.microsoft.com/dotnet/aspnet:8.0` (runtime), compilada com `dotnet/sdk:8.0`

Comandos usados

```
# Local
cp .env.example .env
docker compose up --build -d --wait
docker compose ps
docker compose logs -f api
docker compose down          # mantém os dados (volume)

# Pipeline (job 3) — imagem multi-arquitetura
docker buildx build --platform linux/amd64,linux/arm64 \
  --build-arg APP_VERSION=sha-e4b978c -t ghcr.io/guilhermeeger/cidades-esg-inteligentes:sha-
e4b978c --push .

# Pipeline (jobs 4 e 5) — deploy/deploy.sh <ambiente> <imagem>
docker compose -p residuos-staging --env-file deploy/staging.env pull
docker compose -p residuos-staging --env-file deploy/staging.env up -d --no-build --wait
deploy/smoke-test.sh http://localhost:8081
```

Estratégias adotadas no Dockerfile

Estratégia	Benefício
Multi-stage (SDK → runtime)	Imagem final sem SDK nem código-fonte: menor e mais segura
Cache de camadas	`restore` do `.csproj` antes do código: dependências não são baixadas a cada mudança
Multi-arquitetura	Compila na arquitetura do builder (`$BUILDPLATFORM`) para a de destino (`$TARGETARCH`)
Usuário não-root	`USER $APP_UID` reduz o impacto de uma invasão
HEALTHCHECK	O Docker só considera o container pronto quando API e banco respondem
Versão embutida	`APP_VERSION` = commit, exibida em `GET /`

Dockerfile

```
# syntax=docker/dockerfile:1
# ================================================================
# Cidades ESGInteligentes — API de Gestão de Resíduos (.NET 8)
# Build multi-stage e multi-arquitetura (linux/amd64 e linux/arm64).
# ================================================================

# ---------- Estágio 1: build ----------------------------------------
# O SDK roda na arquitetura da máquina que faz o build ($BUILDPLATFORM)
# e compila para a arquitetura de destino ($TARGETARCH). Assim a mesma
# imagem roda no servidor Linux (amd64) e no Mac Apple Silicon (arm64)
# sem emulação lenta durante a compilação.
FROM --platform=$BUILDPLATFORM mcr.microsoft.com/dotnet/sdk:8.0 AS build
ARG TARGETARCH
WORKDIR /src

# Restaura só o .csproj primeiro: enquanto as dependências não mudarem,
# essa camada vem do cache e o build fica bem mais rápido.
COPY src/ResiduosApi/ResiduosApi.csproj src/ResiduosApi/
RUN dotnet restore src/ResiduosApi/ResiduosApi.csproj -a $TARGETARCH

# Copia o restante do código e publica em Release.
```

```
COPY src/ResiduosApi/ src/ResiduosApi/
RUN dotnet publish src/ResiduosApi/ResiduosApi.csproj -c Release -a $TARGETARCH \
    --no-restore -o /app/publish /p:UseAppHost=false

# ---------- Estágio 2: runtime ----------------------------------------
# Só o ASP.NET runtime (sem SDK nem código-fonte): imagem menor e com
# menos superfície de ataque.
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime

# curl é usado apenas pelo HEALTHCHECK abaixo.
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY --from=build /app/publish .

# Versão (commit) gravada pelo pipeline; aparece em GET /
ARG APP_VERSION=local
ENV APP_VERSION=$APP_VERSION \
    ASPNETCORE_HTTP_PORTS=8080

EXPOSE 8080

# Roda como usuário sem privilégios (já existe nas imagens .NET 8).
USER $APP_UID

# O Docker marca o container como "healthy" só quando a API e o banco respondem.
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=5 \
    CMD curl -fsS http://localhost:8080/health || exit 1

ENTRYPOINT ["dotnet", "ResiduosApi.dll"]
```

4. Evidências: pipeline e ambientes

Prints da execução https://github.com/GuilhermeEger/cidades-esg-inteligentes/actions/runs/38284813510 e dos ambientes publicados por ela.

Job da execução de referência	Resultado	Duração
1. Build	✅ success	51s
2. Testes automatizados	✅ success	26s
3. Imagem Docker (GHCR)	✅ success	33s
4. Deploy Staging	✅ success	23s
5. Deploy Produção	✅ success	19s

4.1 Pipeline rodando, do push à produção

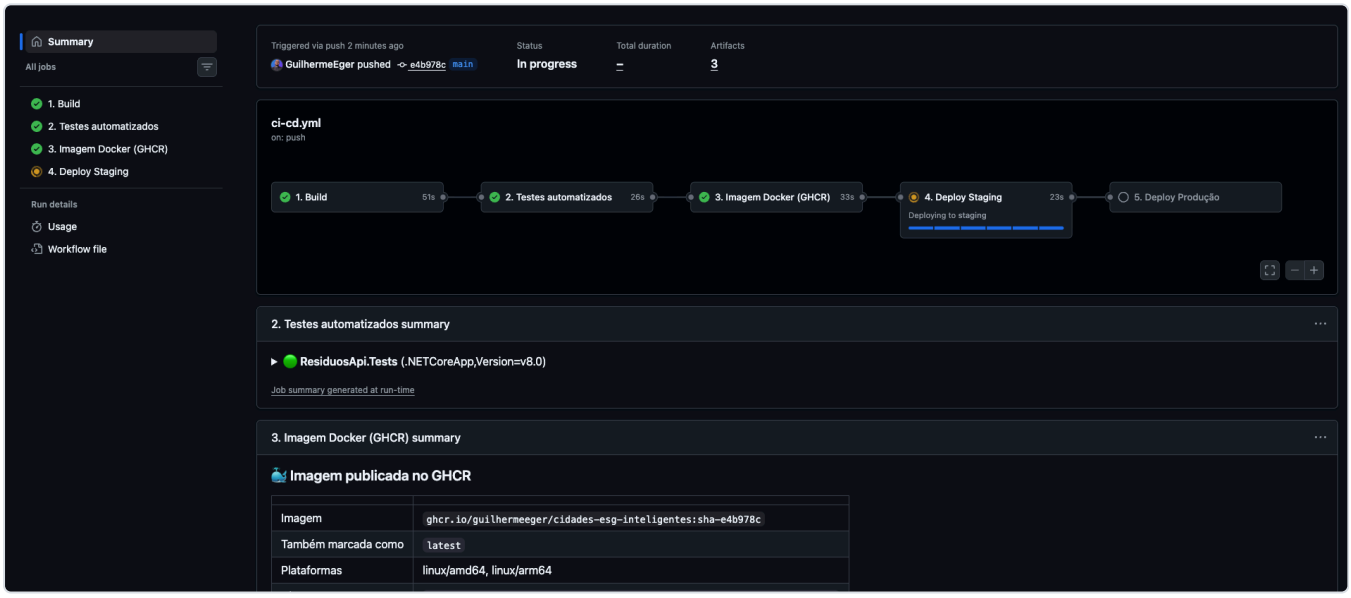


Figura 1. Pipeline em andamento: build, testes e imagem concluídos; deploy em staging rodando no runner self-hosted

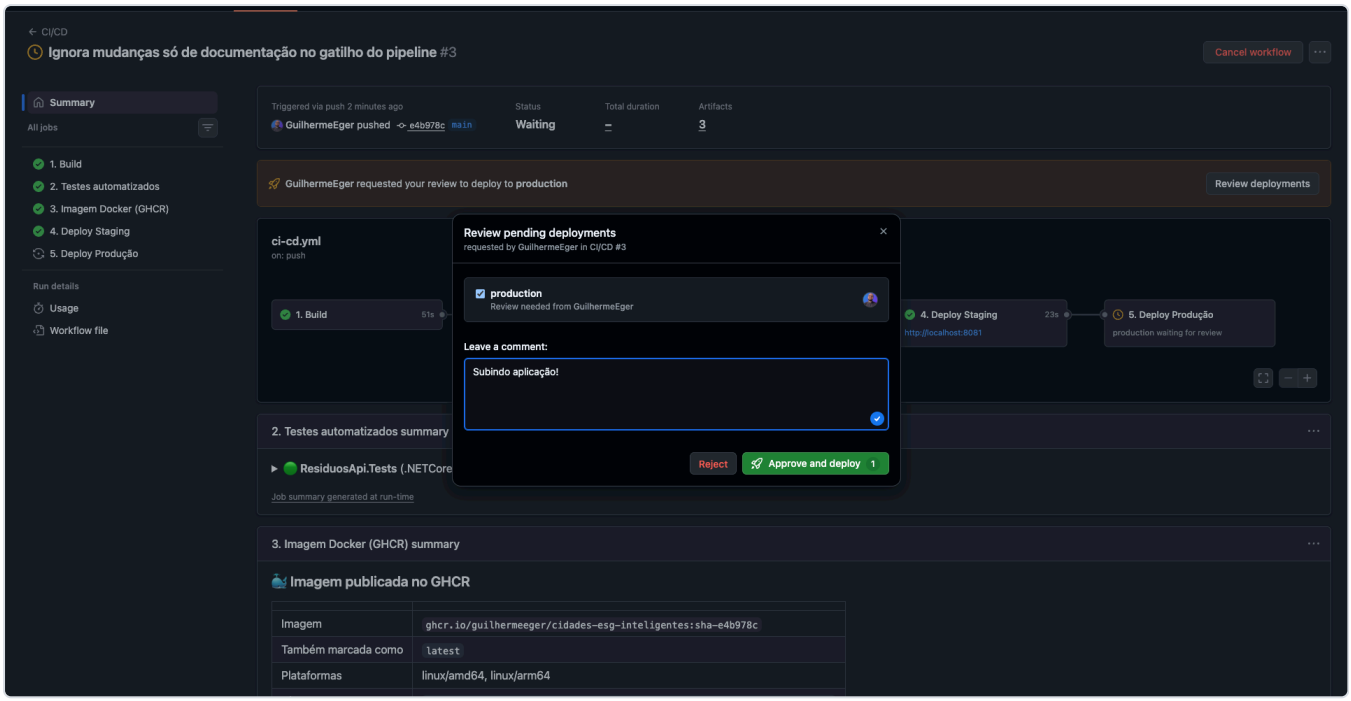


Figura 2. Aprovação manual do deploy em produção (Review pending deployments → Approve and deploy)

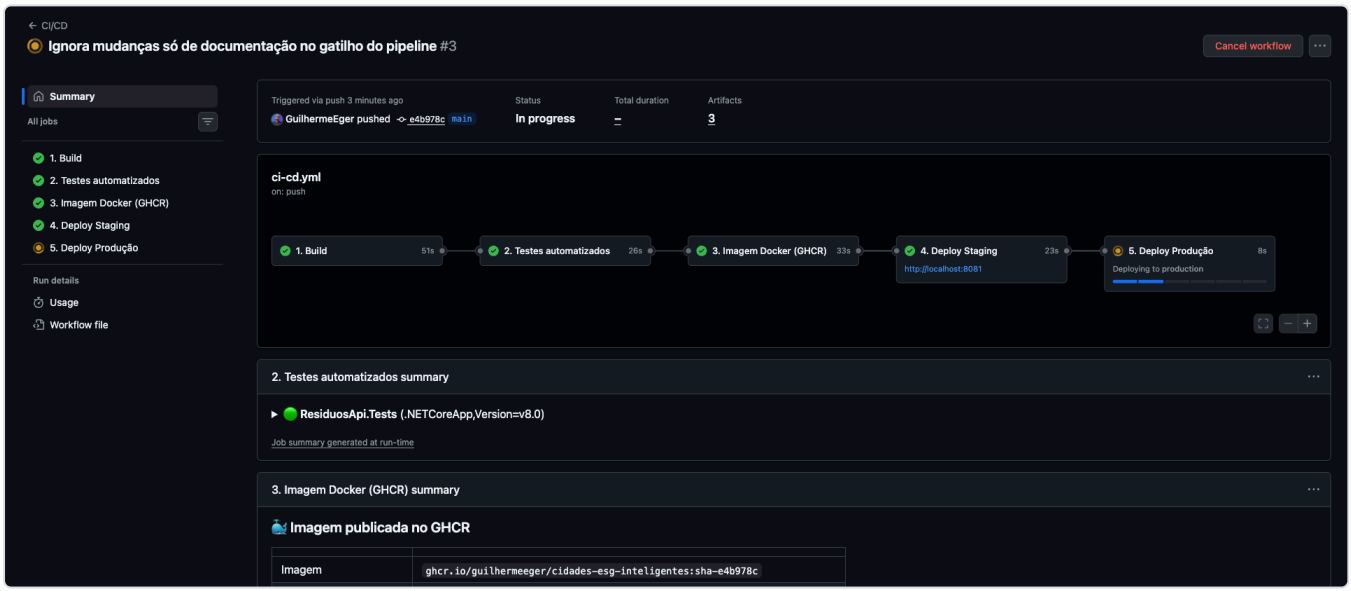


Figura 3. Após a aprovação: deploy em produção em andamento; staging já publicado em http://localhost:8081

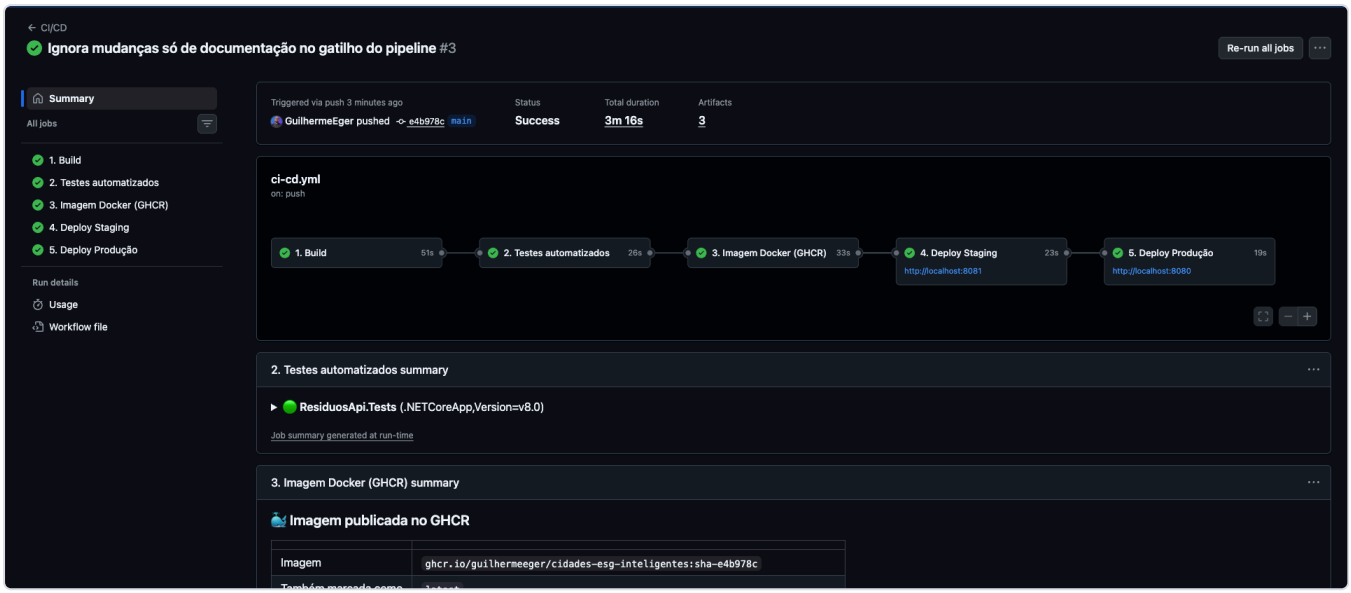


Figura 4. Execução concluída com sucesso: os 5 jobs verdes em 3min16s, com os links dos dois ambientes

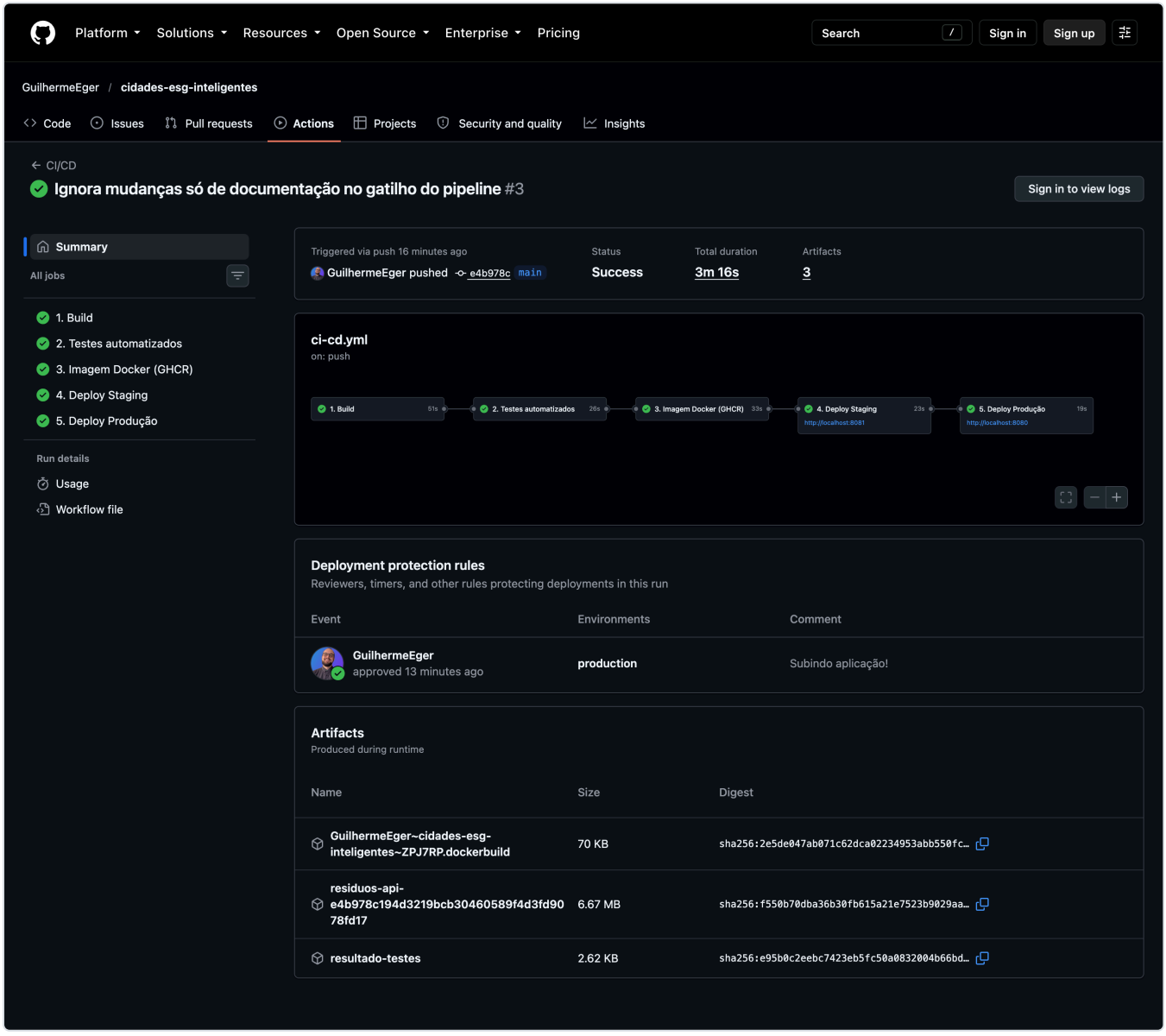


Figura 5. Execução pública do pipeline: 5 jobs, aprovação registrada para production e artefatos gerados

4.2 Build e testes

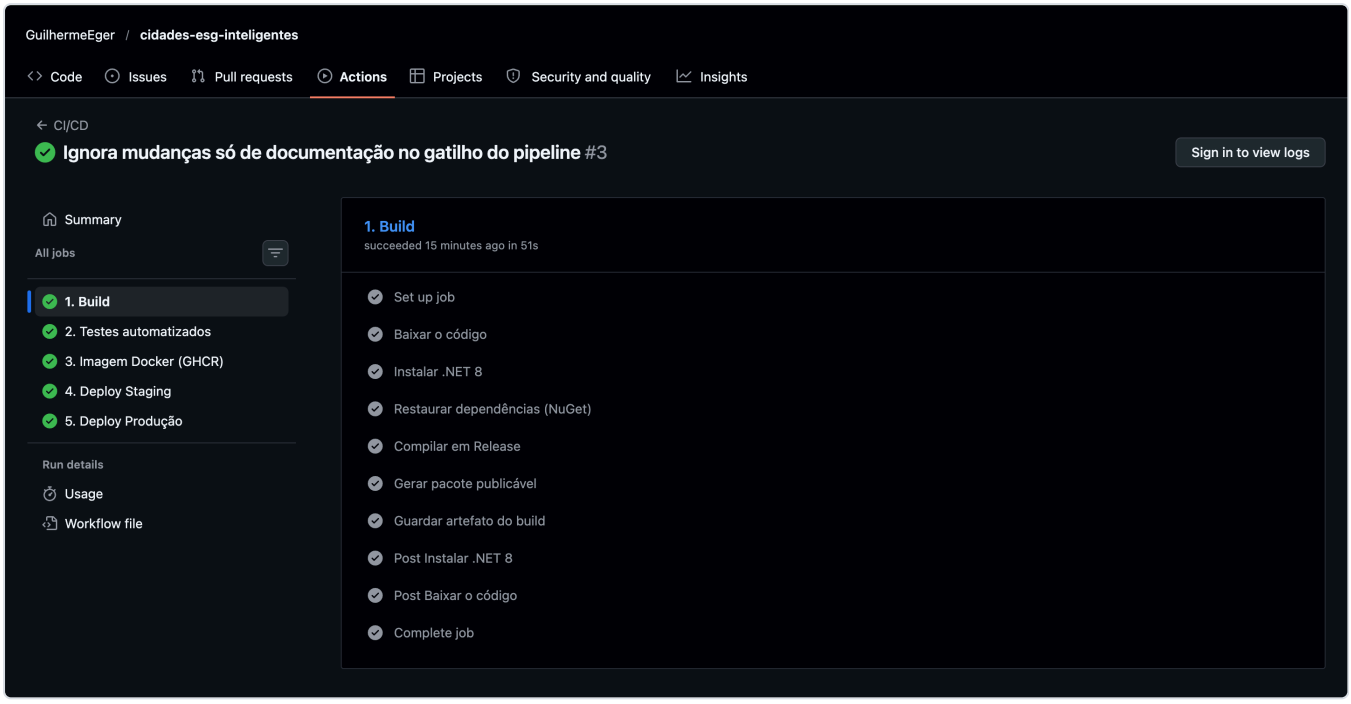


Figura 6. Job 1 — Build: restore, compilação Release, publish e artefato

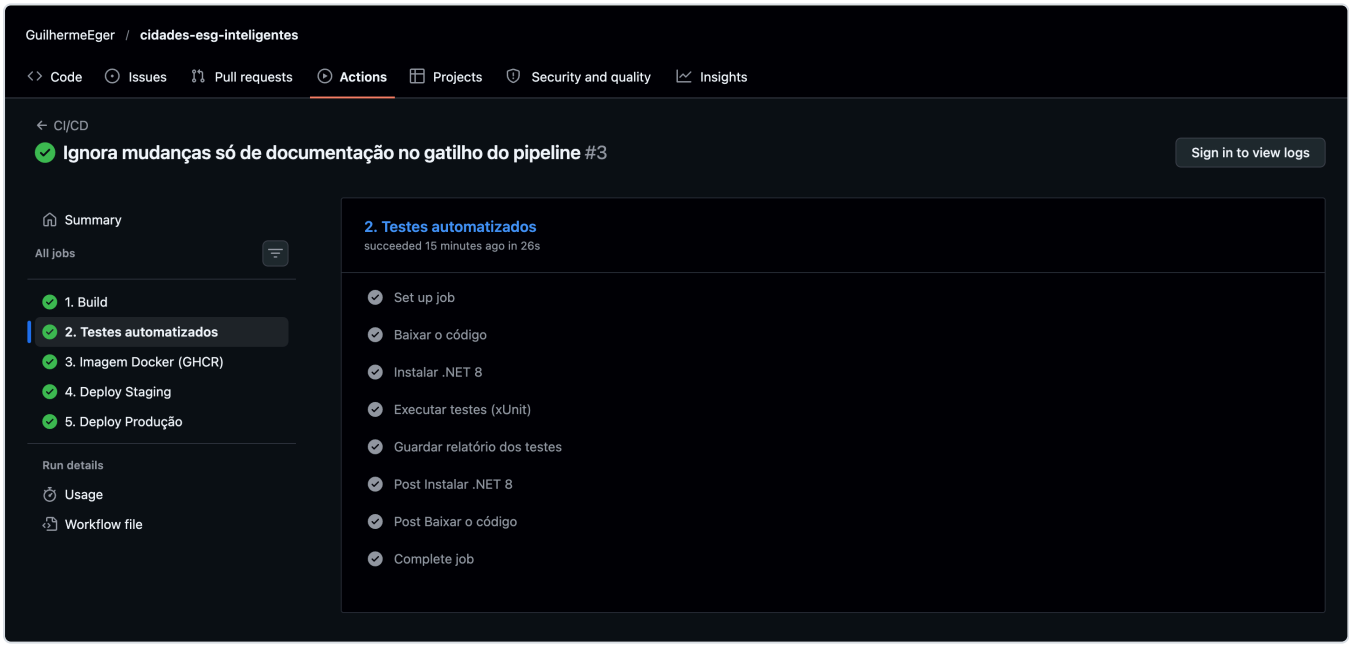


Figura 7. Job 2 — Testes automatizados com xUnit

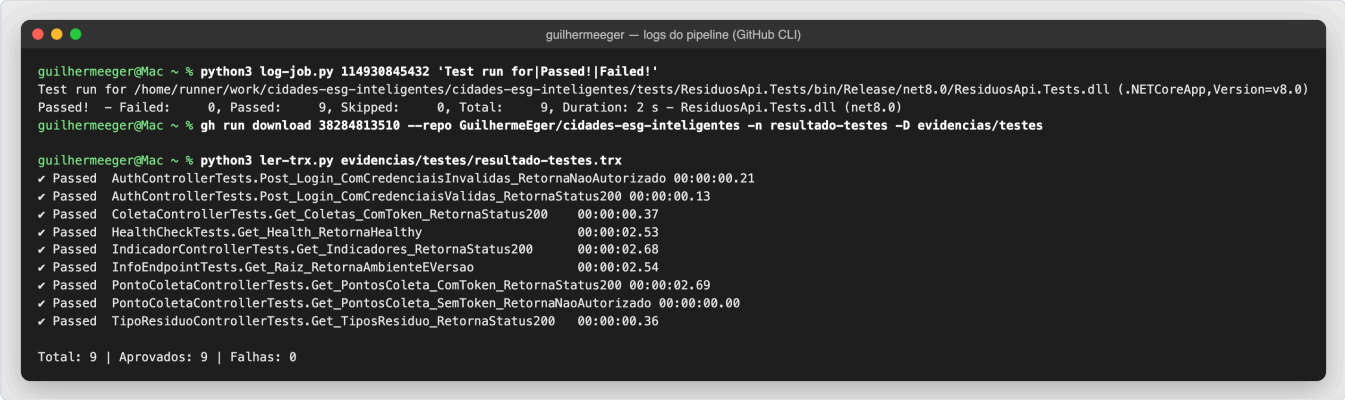


Figura 8. Log do job de testes e relatório .trx baixado dos artefatos: 9 testes aprovados

4.3 Imagem Docker

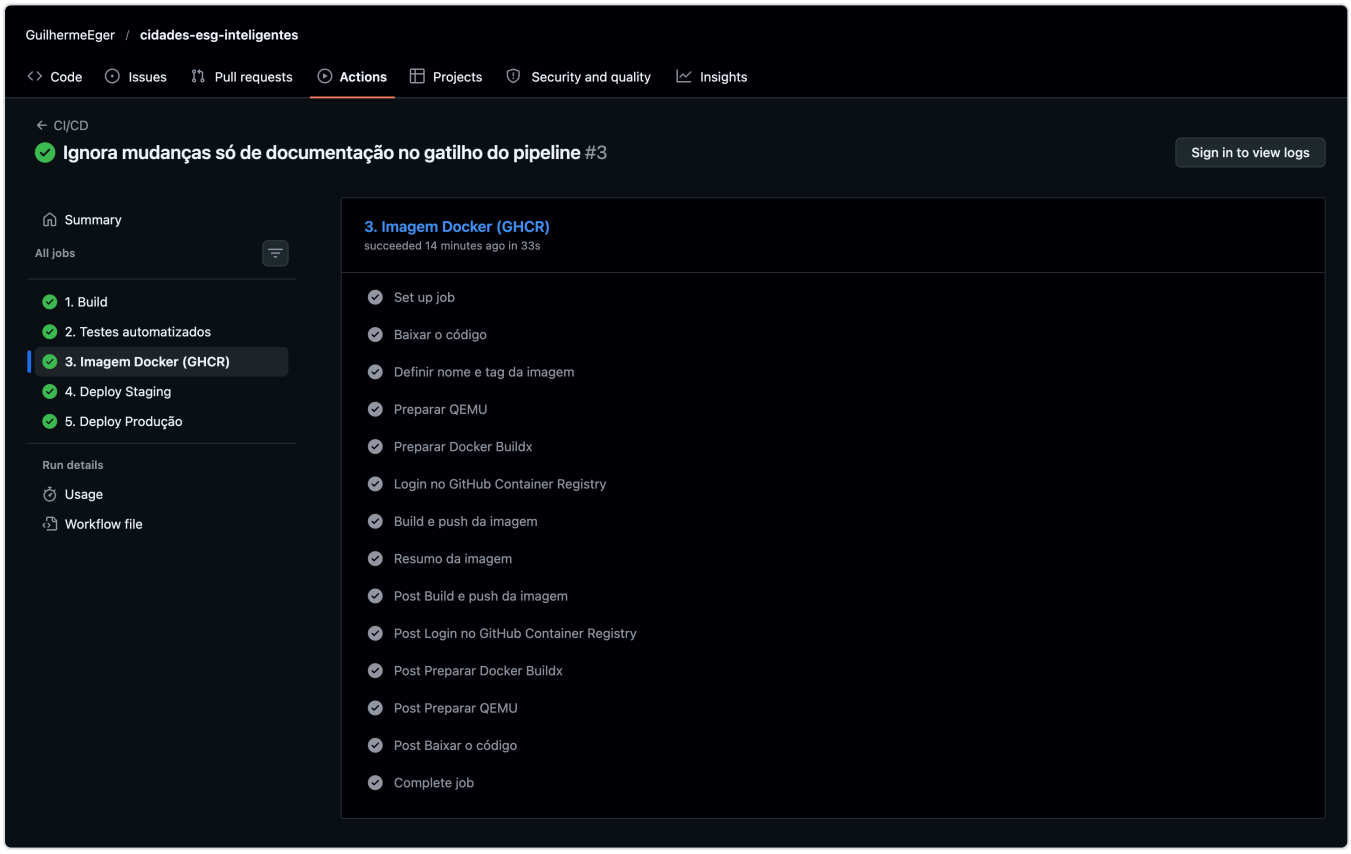


Figura 9. Job 3 — QEMU + Buildx, login no GHCR, build e push da imagem

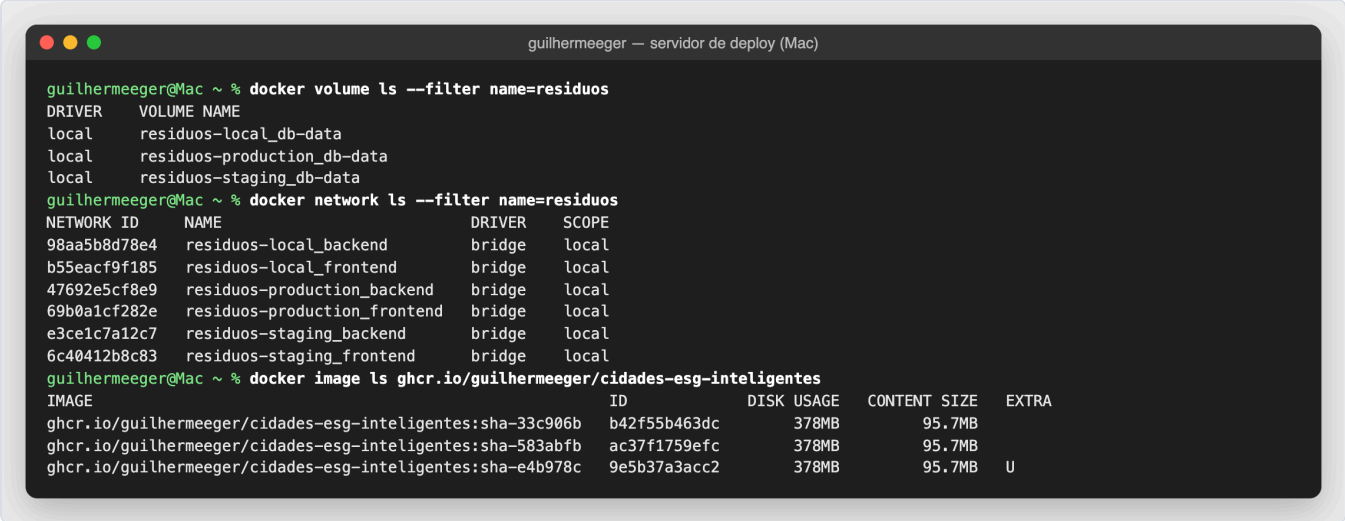


Figura 10. Volumes, redes e imagem publicada pelo pipeline no servidor

4.4 Deploy em staging e produção

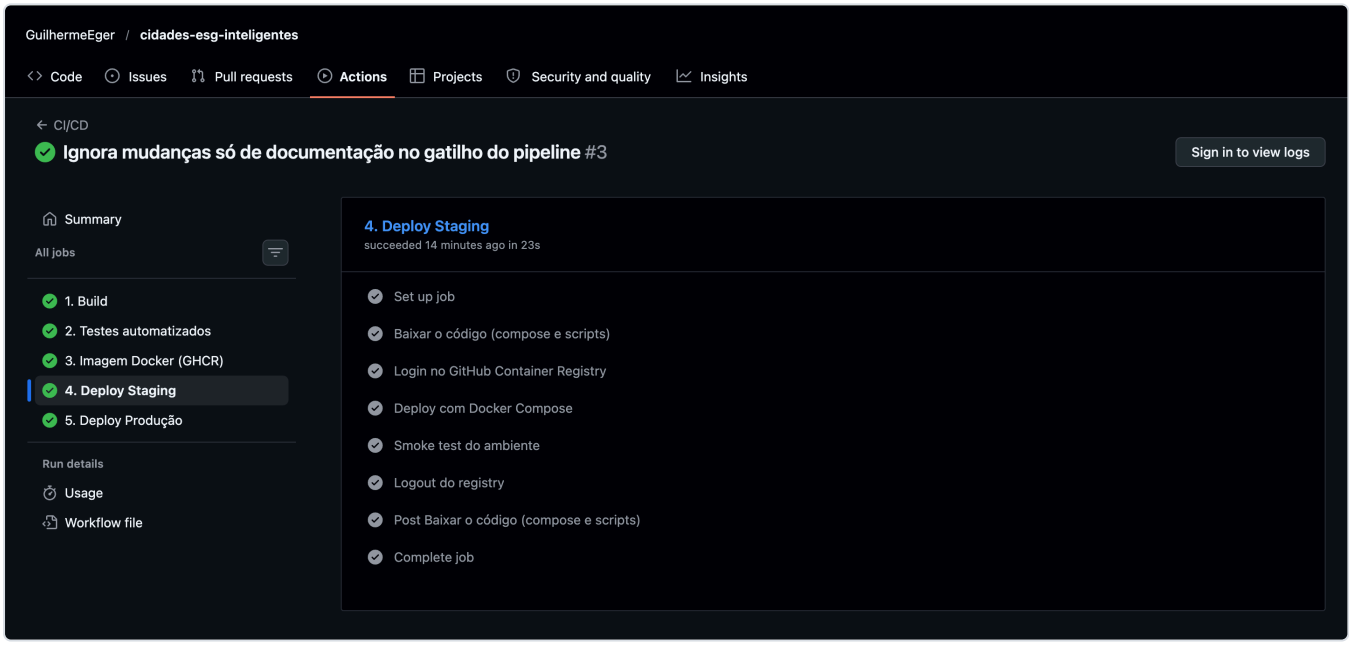


Figura 11. Job 4 — Deploy em staging: login no GHCR, Docker Compose e smoke test

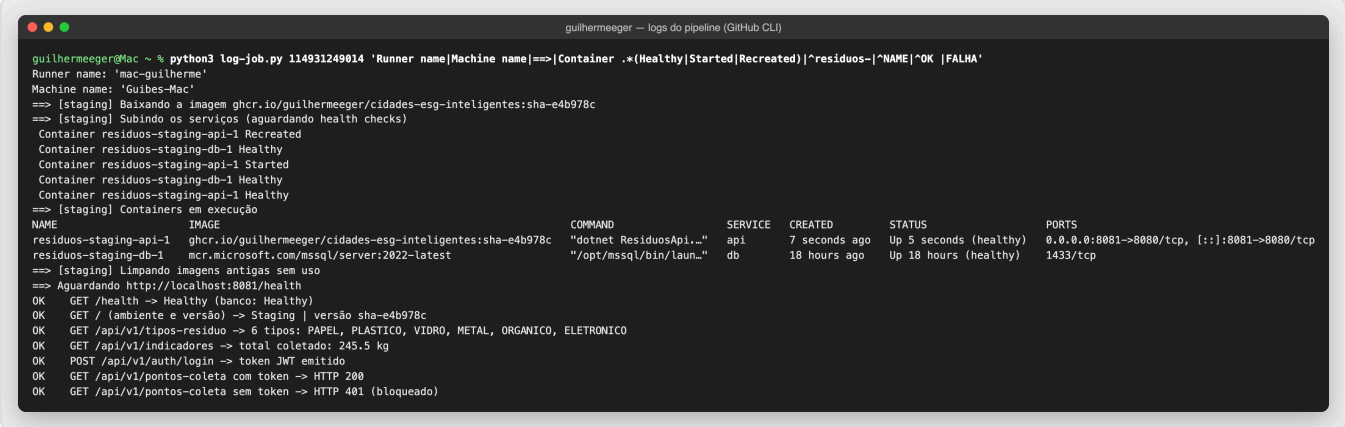


Figura 12. Log do job de deploy em staging: runner self-hosted, Docker Compose e smoke test

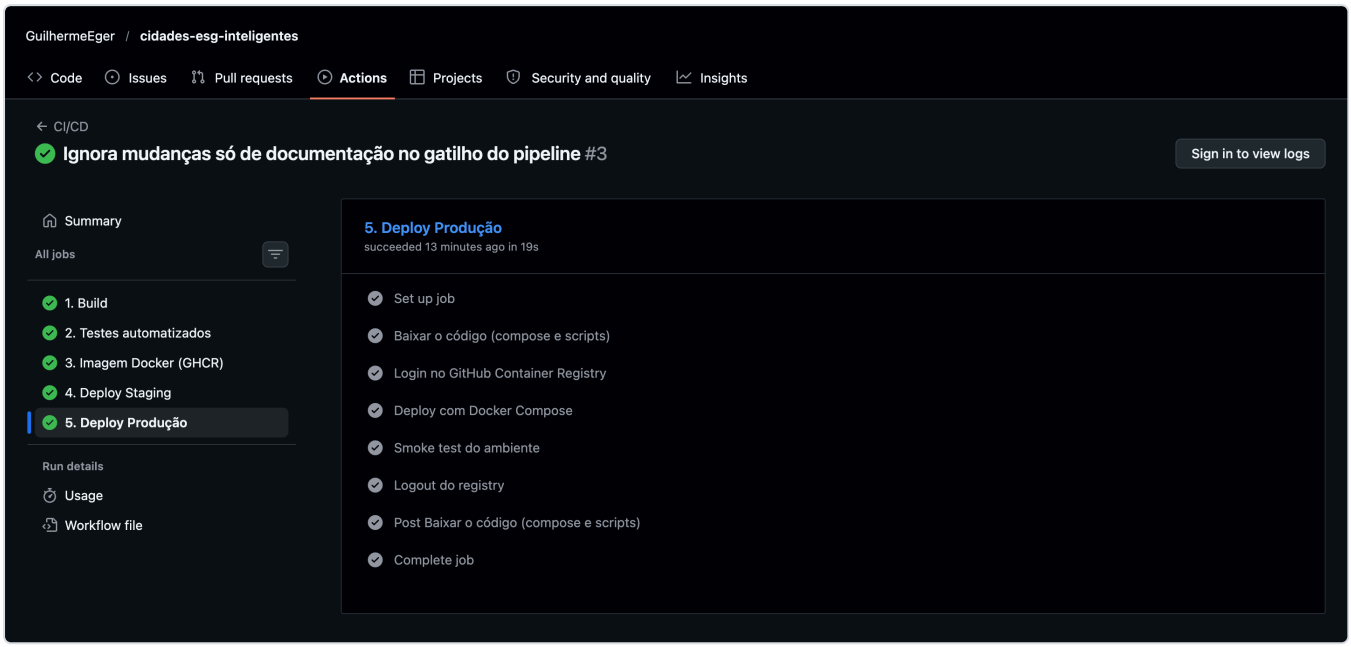


Figura 13. Job 5 — Deploy em produção: mesmos passos, após aprovação manual

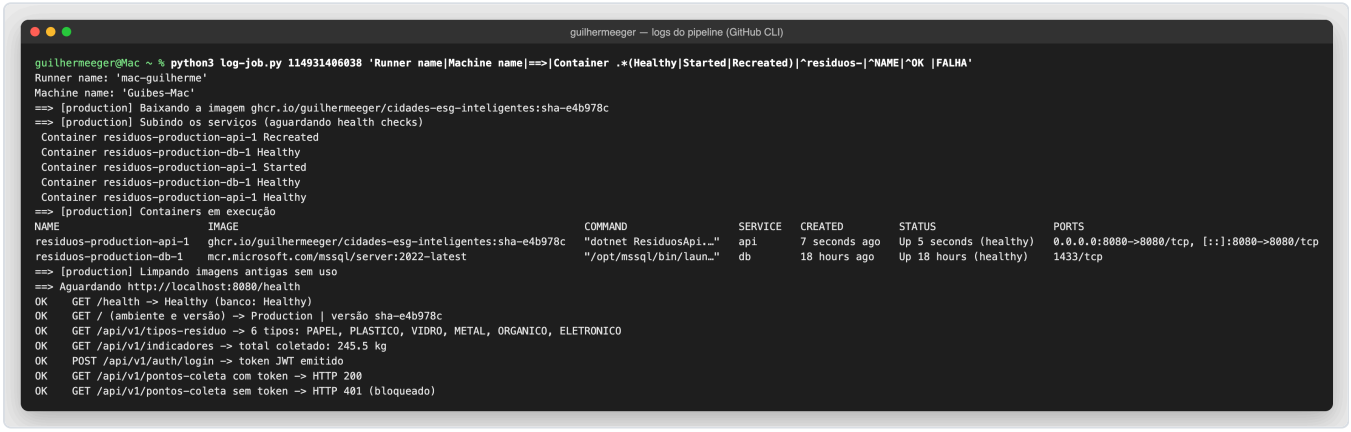


Figura 14. Log do job de deploy em produção: mesma imagem, novo ambiente e smoke test



Figura 15. Environments do GitHub (production com revisor obrigatório) e deploys registrados

4.5 Ambiente de staging funcionando

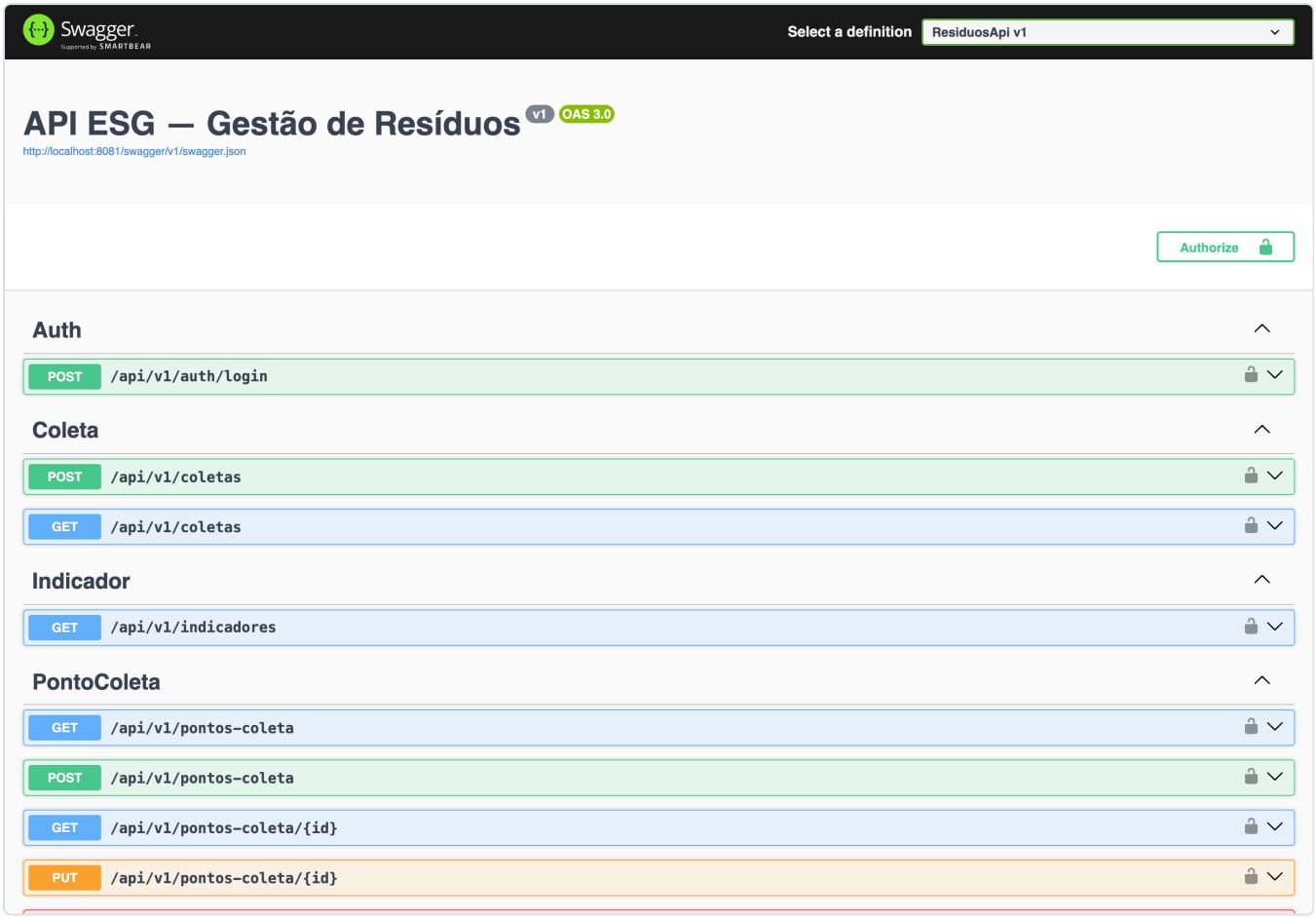


Figura 16. Staging (http://localhost:8081/swagger): Swagger habilitado



Figura 17. Staging respondendo em http://localhost:8081: ambiente, versão, saúde e dados do banco

4.6 Ambiente de produção funcionando

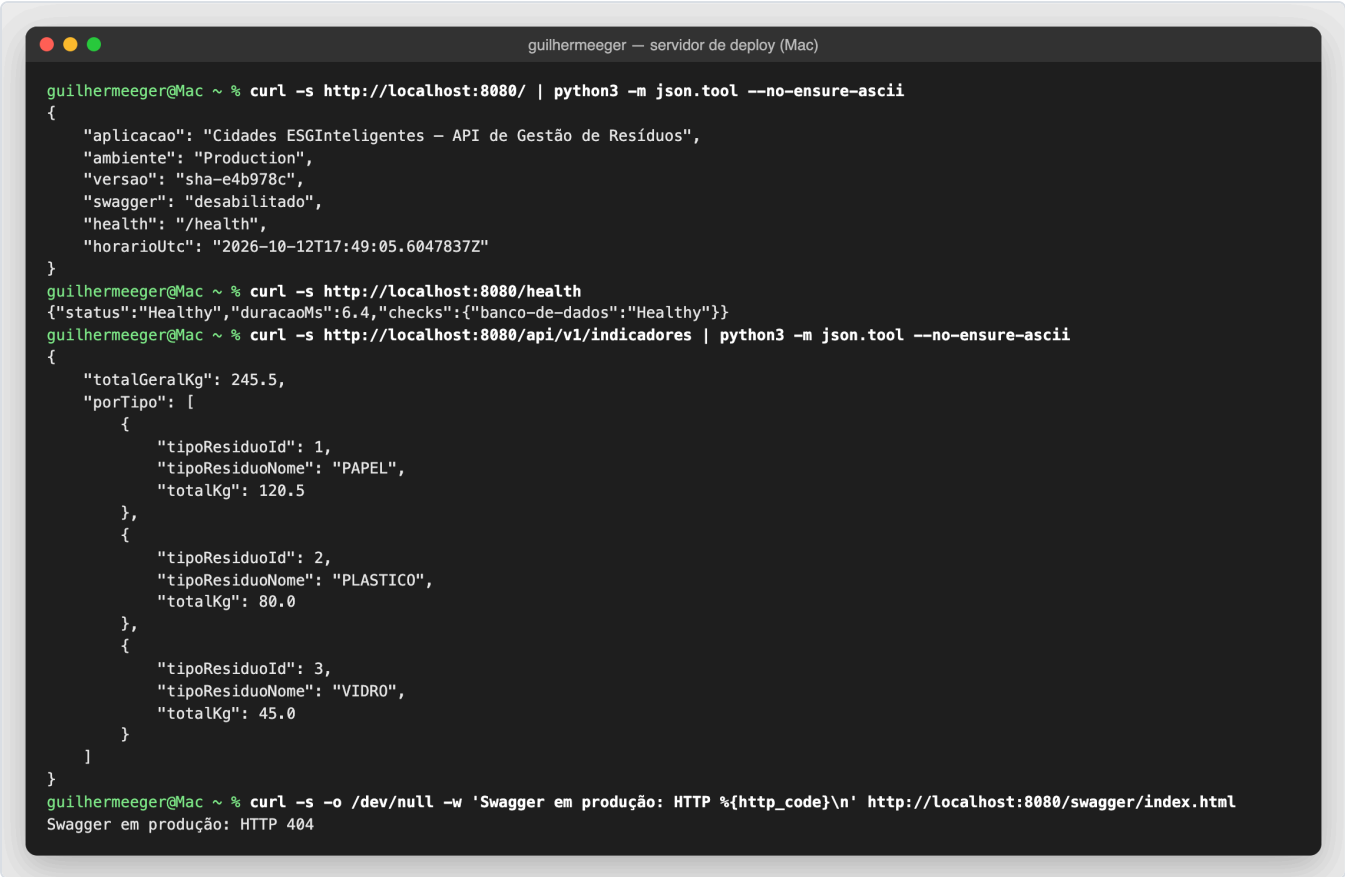


Figura 18. Produção respondendo em http://localhost:8080: mesma versão de staging, banco saudável, indicadores e Swagger bloqueado

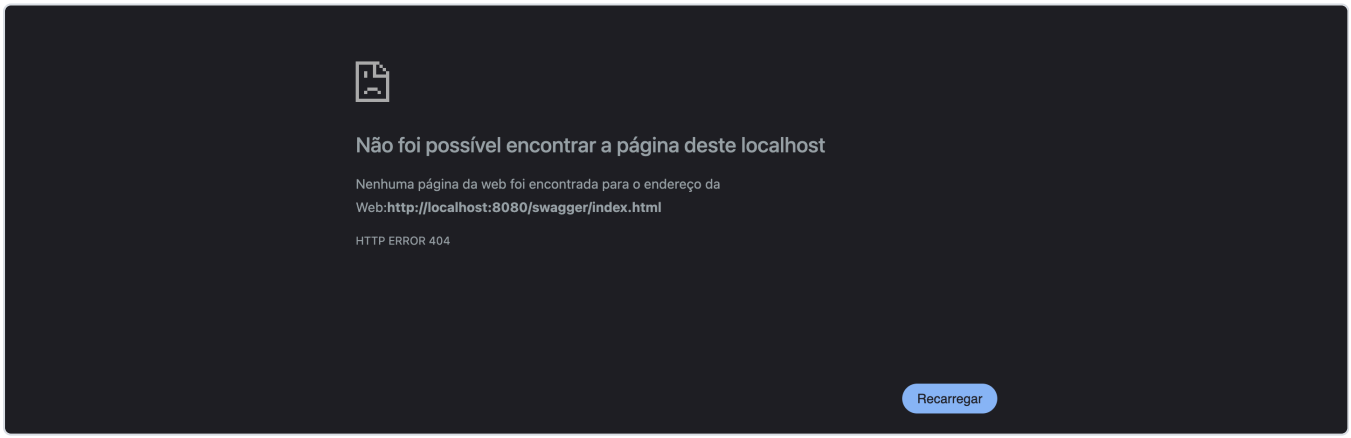


Figura 19. Produção (http://localhost:8080/swagger): Swagger desligado por segurança (HTTP 404)

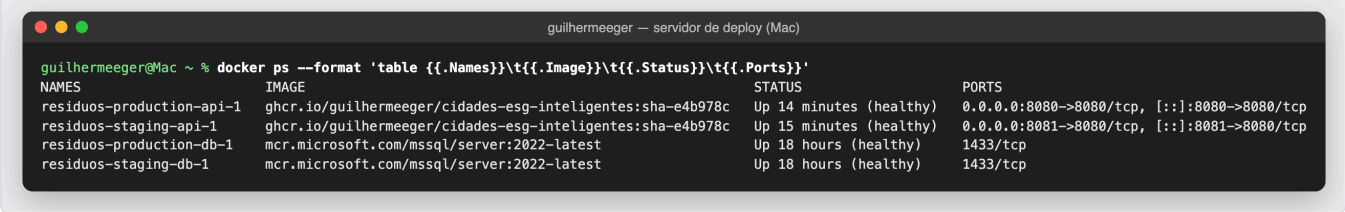


Figura 20. Servidor de deploy: containers de staging e produção (API + banco), todos healthy

5. Desafios encontrados e soluções

SQL Server não tem imagem para Apple Silicon. O servidor de deploy é um Mac ARM e a imagem oficial do SQL Server só existe para amd64. **Solução:** Docker via Colima com virtualização nativa e Rosetta (`--vz-rosetta`) e `platform: linux/amd64` no serviço `db` .

Imagem gerada no GitHub (amd64) x servidor ARM. Solução: build multi-arquitetura com Buildx; o estágio de build roda na plataforma nativa do runner e faz *cross-compile* com `dotnet publish -a $TARGETARCH` , deixando a emulação (QEMU) só para o estágio final.

Staging e produção no mesmo servidor sem conflito. Solução: projetos Compose separados (`-p`), sem `container_name` fixo, portas diferentes (8081/8080) e volumes/redes isolados automaticamente pelo nome do projeto.

A API subia antes do banco aceitar conexões. Solução: healthcheck com `sqlcmd` no banco, `depends_on: condition: service_healthy` , `EnableRetryOnFailure` no EF Core e `docker compose up --wait` no deploy.

A imagem do ASP.NET não tem `curl` para o HEALTHCHECK. Solução: instalação mínima do `curl` no estágio de runtime, com limpeza do cache do apt.

Senha do banco fixada no primeiro start. O SQL Server só aplica a senha do usuário `sa` quando o volume é criado. Um ensaio local com outra senha deixaria o deploy sem acesso ao banco. **Solução:** o ensaio foi removido com `docker compose down -v` antes do primeiro deploy, e cada ambiente tem seus próprios secrets fixos no GitHub.

Segredos fora do código. Solução: secrets por Environment no GitHub, injetados como variáveis no job; o Compose falha com mensagem clara se faltar algum (`${VAR:?}`); `.env` no `.gitignore` e `.env.example` documentado.

Runner self-hosted em repositório público. Um pull request malicioso poderia tentar executar código no servidor. **Solução:** jobs de deploy só em `push` na `main` , aprovação obrigatória para PRs de qualquer colaborador externo e runner ligado apenas quando necessário.

Avisos de depreciação no pipeline. A primeira execução terminou com 7 anotações: as actions usadas rodavam em Node.js 20 (em descontinuação no GitHub) e o rótulo `ubuntu-latest` estava prestes a migrar para o Ubuntu 26. **Solução:** atualização para as versões com Node.js 24 (`checkout@v7` , `setup-dotnet@v6` , `upload-artifact@v7` , actions Docker v4/v7) e runner fixado em `ubuntu-24.04` , o que deixa o ambiente de build previsível. A execução seguinte rodou sem nenhum aviso.

GHCR exige nome de imagem em minúsculas. O usuário do GitHub tem maiúsculas (`GuilhermeEger`). **Solução:** o nome é gerado com `${GITHUB_REPOSITORY,,}` no job de imagem.

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑
Dockerfile funcional	☑
docker-compose.yml ou arquivos Kubernetes	☑
Pipeline com etapas de build, teste e deploy	☑
README.md com instruções e prints	☑
Documentação técnica com evidências (PDF ou PPT)	☑
Deploy realizado nos ambientes staging e produção	☑

Arquivos entregues

`cidades-esg-inteligentes-RM97069.zip`	Código-fonte, Dockerfile, docker-compose.yml, pipeline (`.github/workflows`), scripts de deploy, `.env.example` , README e prints
`FIAP-Fase6-DevOps-GuilhermeEgerPereira-RM97069.pdf`	Este documento
Repositório	https://github.com/GuilhermeEger/cidades-esg-inteligentes